TRAIN SIM CLASSIC · VEHICLE WORKFLOW GUIDE

Part 7

Removing workflow friction: the TSC Toolkit

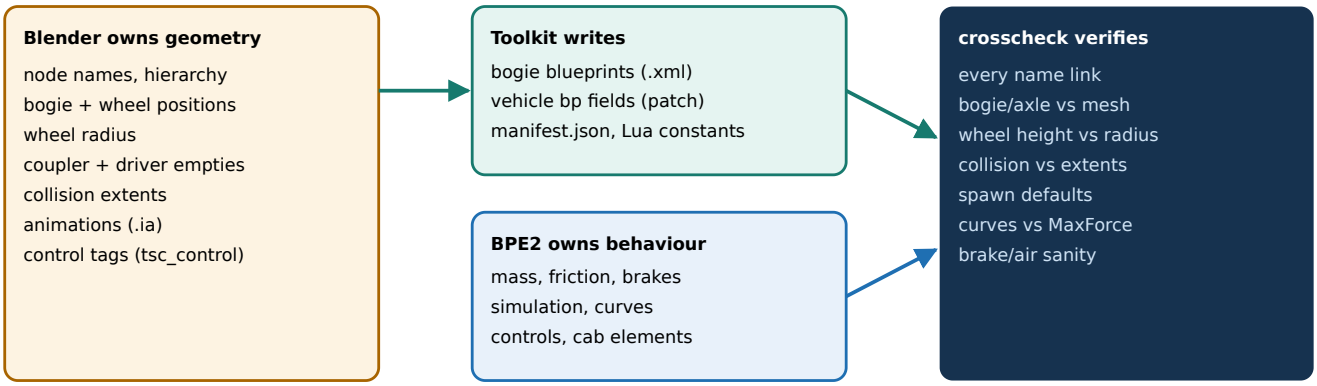
Three tools ship with this guide: a Blender add-on that validates the scene and writes blueprint data from the model, a command-line cross-checker for blueprints against exported geometry, and an offline Lua test harness. Plus ideas for further scripts.

In this part

1. The principle: one source of truth per kind of data
2. Installing the Blender add-on
3. Tags and empties the add-on reads
4. Validate
5. Name
6. Measure + write bogie XML, Patch vehicle blueprint
7. IA jobs
8. Lua skeleton
9. tsc_texture_split.py: oversized textures into tiles
10. tsc_crosscheck.py
11. Offline Lua testing
12. Further scripts worth building (concepts)
13. The daily loop with the tools

Files: `tools/blender/tsc_toolkit.py` and `tools/blender/tsc_texture_split.py` (Blender 4.2+ add-ons), `tools/tsc_crosscheck.py` (Python 3, standard library only), `tools/lua_mock/run_trolley_test.py` (Python 3 + `lupa`). The add-on's logic was exercised headless in Blender 4.2 (bpy) on a synthetic locomotive; the cross-checker was run on the supplied Kuju samples. Test in your own Blender before relying on them.

1 · The principle: one source of truth per kind of data



Geometry facts flow one way: Blender → blueprint. Nothing geometric is typed by hand twice.

Figure 7.1 · Ownership. The toolkit exists because the blueprint has the final say in game while the mesh is where you see the truth. Generating the blueprint side from the mesh removes the disagreement at its source.

2 · Installing the Blender add-on

1. Edit ▸ Preferences ▸ Add-ons ▸ (drop-down) Install from Disk… ▸ choose `tools/blender/tsc_toolkit.py`. Enable “TSC Toolkit”.
2. In the 3D Viewport press `N` ▸ tab **TSC**.
3. Set **Output folder** to the vehicle's Source folder (e.g. `…\Default\Bogies` for bogie XML) and, for patching, pick the **Vehicle blueprint** .xml.

Requires Blender 4.2 or later (layered actions from 4.4 are supported). If you export with BRIAGE from a Blender 2.79 install, run the toolkit in your modelling Blender before transferring the model.

3 · Tags and empties the add-on reads

Tag / object	Meaning
Objects named `…boNN`	Bogies (bo01 front). Their children named `…boNNwhNN` are wheelsets.
Empty `tsc_coupling_front`	Front coupler point; its Y axis points forward. Becomes `FrontCouplingPivot`.
Empty `tsc_coupling_rear`	Rear coupler, rotated 180° about Z. Becomes `RearCouplingPivot`.
Empty `tsc_driver`	Driver position (`DriverPosition`).
Custom property `tsc_ia` = "sample_throttle"	This object's animation is exported as `sample_throttle.ia`.
Custom property `tsc_control` = "Regulator"	This cab object drives that control (used for the Lua constants and the manifest).
`tsc_min`, `tsc_max`, `tsc_default`	Control range, recorded in the manifest for reference.

The `tsc_` empties are skipped by the naming check. Keep them in a collection you exclude from BRIAGE's export (or hide them if you use “Export visible”).

4 · Validate

Writes a report to the Text Editor block **TSC_Validation** and a summary in the status bar. Checks:

- Name prefix `L_DDDD_` present; `.00N` suffixes; length > 24 (warn) / > 31 (error); spaces.
- LOD chains: parent is LOD n−1, same base name, larger distance.
- Negative scale (error), unapplied scale (warn).
- More than 24 children on a node; more than 256 exportable objects.
- bo01 at +Y, bo02 at −Y; each bogie has `boNNwhNN` wheels.
- Each wheel's lowest point is on Z = 0 (catches the sample's 0.525 vs 0.55 problem).
- Main body origin at 0,0,0.
- Actions that start after frame 0 (rest pose missing); non-linear keys.
- Animated root objects without a `tsc_ia` tag.
- Meshes without UV map or material; empty material slots; slots used by no face (the sample's stray `f1`).
- Image names with extra dots; images on a different drive from the .blend.

```
ERROR 2_0400_engine: LOD distance 400 must be greater than parent's 500
WARN  Handrail: no L_DDDD_ prefix - BRIAGE will rename it 1_1000_Handrail
WARN  Handrail: unapplied scale (1.0, 2.0, 1.0) (IA files carry no scale; apply it)
WARN  1_0000_throttle: action '1_0000_throttleAction' starts at frame 1; BRIAGE exports the IGS
at frame 0 - key the rest pose on frame 0
WARN  1_0500_engine: material slot 1 'f1' is not used by any face (it will still export)
```

Real output from the headless test scene, which deliberately contained these mistakes.

5 · Name

Set **LOD** and **Distance**, select objects, press **Apply LOD name to selected**. The base name is kept, any old prefix and `.00N` suffix are removed: `Handrail.003` → `1_1000_Handrail`.

6 · Measure + write bogie XML, Patch vehicle blueprint

Measure + write bogie XML

Reads the model and writes `tsc_manifest.json` plus one bogie blueprint per `boNN` (e.g. `bo01.xml`) in Kuju's source format: wheel radius measured from each wheel's vertical extent, axle offsets relative to the bogie origin, node names, gauge 1.435, stiffness/damping as the sample, `Powered` on (option for the rear bogie).

```
{ "index": 1, "node": "bo01", "pivot_x": 5.583, "pivot_y": 0.27,
  "axles": [ { "node": "bo01wh01", "offset": 2.217, "radius": 0.525, "centre_height": 0.525 },
             { "node": "bo01wh02", "offset": 0.0,   "radius": 0.525, "centre_height": 0.525 },
             { "node": "bo01wh03", "offset": -2.206,"radius": 0.525, "centre_height": 0.525 } ] }
"collision": { "CollisionCentreX": 0.0, "CollisionCentreY": 2.2515, "CollisionLength": 19.124,
               "CollisionWidth": 2.856, "CollisionHeight": 4.563 }
"couplings": { "FrontCouplingPivot": [1,0,0,0, 0,1,0,0, 0,0,1,0, 0,1.025,9.14,1],
               "RearCouplingPivot":  [-1,0,0,0, 0,1,0,0, 0,0,-1,0, 0,1.025,-9.14,1] }
```

Manifest excerpt from the test scene built to the sample loco's dimensions. The rear coupler matrix reproduces the Kuju sample's (−1, 1, −1) diagonal exactly, confirming the axis conversion.

Patch vehicle blueprint

Writes into the chosen engine/wagon blueprint: the five `Collision*` fields, each `Bogey[i].BogeyPivotX` (bogies matched in order, bo01 first), `FrontCouplingPivot`, `RearCouplingPivot` and `DriverPosition`. It updates both the real value and any ToolData "typed in" value, keeps a `.bak`, and lists every change in the text block **TSC_Patch**.

> **Limits**
>
> - Close the blueprint in BPE2 first (or reload it afterwards). BPE2 may re-save its own formatting; that is fine.
> - The collision box is the bounding box of LOD 1 meshes excluding lights, shadows, digits, pantographs and wipers. Review it; you may want it shorter than the buffers, as Kuju did.
> - Receiving points are not patched (the samples place them 0.1 m beyond the pivots); add empties and extend the script if you need them.

7 · IA jobs

Tag each animated root object with `tsc_ia`. **Prepare next IA job** cycles through them alphabetically: selects the object, sets the frame range to its action (from 0), jumps to frame 0 and copies `name.ia` to the clipboard. Then press BRIAGE's IA Export and paste the name. Twelve cab animations become twelve clicks of the same two buttons.

If you know your BRIAGE build's IA operator id (enable Preferences ▸ Interface ▸ Python Tooltips and hover the export button), type it into **BRIAGE IA operator**; the toolkit then calls it with `filepath=` and exports without the manual click. `check` the operator's parameter names on your build.

8 · Lua skeleton

Writes `<blend name>_EngineScript.lua` (never overwrites) with a constant per tagged control, the pass-through `OnControlValueChange` (with the correct `== 1` test), `Initialise` calling `BeginUpdate`, and an `Update` gated by `GetIsPlayer`.

```
CTRL_REGULATOR = "Regulator"
CTRL_TROLLEYPOLEFRONT = "TrolleyPoleFront"
```

9 · tsc_texture_split.py: oversized textures into tiles

The second add-on in the TSC tab (panel **Texture split**). It implements Part 2 §7.1: a material whose images are larger than the tile size is replaced, on the selected mesh objects, by one material per used tile, with the mesh cut along the tile borders in UV space and the UVs remapped. Cropping, not baking: lossless and seconds, not minutes.

Setting	Meaning
Output folder	Where the tile images go (your Textures directory). Names: `<image>_c<col>r<row>.tga`, one dot, as BRIAGE wants.
Tile width / height px	2048 × 2048 by default. Any size works (the test used 1920 × 1080 too); the game wants powers of two.
Border inset (texels)	0.5: UVs stop half a texel short of the tile edge so bilinear filtering never wraps to the opposite side. 0 for a mathematically exact crop.
Pad partial tiles	When the sheet does not divide evenly, the last column/row tiles are written at their real size (e.g. 256 × 1080) or padded to the full tile.
Format	TGA or PNG; ACE conversion is the same external step as before.
Only material	Restrict to one material name; empty = every oversized material on the selection.
Remove emptied slots	Drop the original material slot once no face uses it (the material itself stays in the file).

1. Work on the export copy, not the master (the operator edits the mesh; undo works, but keep the master clean).
2. Select the objects; set the folder and tile size; press **Split oversized textures into tiles**.
3. Read the Text Editor block `Texture_Split`: grid, number of faces cut, tiles used. The JSON report in the output folder lists material → tile → images.
4. Export with BRIAGE as usual: one material per tile, same shader settings (the tile materials are copies, including custom properties and node setup; every image node is swapped for its crop, repeat mode set to extend).

How the cut works: for each tile line, every face whose UVs straddle it is triangulated if needed, its crossing edges are split at the exact crossing (the new loop UVs are interpolated per adjacent face, so UV seams survive), and the two new vertices are connected. Vertices are shared across the cut, so smoothing and watertightness are kept; the only change is the material on each side. Faces whose UVs repeat beyond 0..1 are handled (the lines are generated across the mesh's UV range and tiles wrap).

Tested with `tools/tsc/texture_split_test.py`: a 4096 colour grid on a 310-face body became 586 faces in four 2048 tiles; 600 random surface points mapped back to the source position within 0.001 texel; render difference mean 0.0014; boundary edge count unchanged; a second map in the same material was cropped identically; custom properties survived the material copy; the 1920 × 1080 request produced 3 × 4 tiles, padded and unpadded. **verified** Not run through BRIAGE or BPE2 here: the next check is one split object exported to an .igs and converted. **check**

10 · tsc_crosscheck.py

```
python tools\tsc_crosscheck.py --engine Sample_Engine.xml --sim "Electric Engine Simulation.xml"
^
    --bogies "Sample Bogie 01.xml" "Sample Bogie 02.xml" ^
    --geo sample_engine.geo.xml --cabgeo sample_cab.geo.xml --csv TractiveEffortVsThrottle.csv
```

Real result on the Kuju sample electric (abridged):

```
==== ERROR (2) ====
  [cab] control EngineStart -> node 'start' (cInteriorPushButton) not found in cab geometry
  [cab] control PantographControl -> node 'hs_aus_lit' (cInteriorVisibilityObject) not found in
cab geometry
==== WARN (11) ====
  [control] TrainBrakeControl default 0.70 - vehicle spawns with brakes applied
  [bogie] bogie 'bo01' modelled at 5.583 m but blueprint pivot is 6.250 m (difference 0.667 m)
  [axle] 'bo01wh01' wheel centre is 0.525 m above rail origin but radius is 0.550 m (0.025 m sunk
into rail)
  … (same for the other five wheelsets)
  [body] collision length 18.00 differs from mesh length 19.12 by 1.12 m
  [body] collision box top 4.00 m is below mesh top 4.53 m (roof/pantograph not covered)
  [body] MaxPermissibleSpeed is 0.0 - AI and HUD will have no vehicle speed limit
==== OK (15) ====
  [cab] control Regulator -> cab node 'throttle' found   … and 14 more links
  [brake] brake CabControlName 'TrainBrakeControl' matches a ControlValue
```

Checks performed: duplicate control names; defaults outside min/max; spawn-braked default; every pick node, visibility transform and exterior AnimationID; bogie and axle nodes exist, positions and radii; collision box vs extents; coupling points; `MaxPermissibleSpeed`; MaxForce vs curve peaks; throttle curve origin; MaxContinuousForce ≤ MaxForce; `CabControlName` links; empty compressor; zero cut-out pressure; empty loco brake (info). Exit code 1 on any error, so it can gate a batch build.

11 · Offline Lua testing

`tools/lua_mock/run_trolley_test.py` loads a script into a Lua runtime (`pip install lupa`), replaces `Call` / `SysCall` with a Python mock holding control values, speed, curvature and animation times, and steps `Update` at 60 Hz. Copy it for your own scripts: mock only the calls you use, script the player's inputs, assert on control values and alerts. It turns a five-minute game round-trip into a one-second test.

12 · Further scripts worth building (concepts)

Idea	What it would do	Friction removed
LOD generator	For selected LOD 1 objects, duplicate, add a Decimate modifier at a chosen ratio, rename to `2_DDDD_base`, parent correctly.	Hand-building LOD twins

Idea	What it would do	Friction removed
Shadow mesh generator	Convex hull (or decimated copy) per selected part, named `1_0400_shadow_base`, `shadow_` material with StencilShadow.fx.	Shadow modelling
Control previewer	A custom property `tsc_value` 0–1 on each cab object, with a driver mapping it to the action frame, so you can check levers and needles at any value before export.	Export-to-check loops
Custom keyword writer	Collect every object tagged `tsc_ia` (and its animated children) and write them into BRIAGE's Custom keywords table so animated parts are never merged.	Silent merge bugs
Texture pipeline	Check power-of-two sizes, rename to a consistent scheme, collect into the Textures directory, warn on alpha where the shader ignores it.	Texture path errors
Number-plate builder	Generate a `primarydigits_N` plane with N faces and `primarynumber_i` materials from a picked rectangle.	Fiddly UV/material set-up
Control-value generator	From `tsc_control` tags, write the `ControlValues` XML (name, range, lever element, pick node, .ia path) for BPE2.	Typing controls twice
Round-trip diff	Read a serz-converted GEO and compare node names, positions and materials against the scene (what the supplied scene read did by hand).	“What actually got exported?”

13 · The daily loop with the tools

1. Model / animate in Blender; tag cab objects (`tsc_ia`, `tsc_control`).
2. **Validate** until no errors.
3. BRIAGE IGS export; **Prepare next IA job** → BRIAGE IA export, per animation.
4. **Measure + write bogie XML**, then **Patch vehicle blueprint**.
5. BPE2: behaviour fields, controls, simulation; export.
6. Convert the exported geometry with serz and run **tsc_crosscheck.py**; fix errors.
7. Game test drive (Part 6 order). Script changes: test offline with the Lua mock first.